

CHAPTER THREE

RESEARCH METHODOLOGY

3.1 Introduction

This chapter provides a granular technical breakdown of the architectural framework and procedural paradigms utilized in the engineering of the **Agri-Go** platform. It sheds light on the **Software Development Life Cycle (SDLC)**, the **System Architecture**, the **Data Modeling** strategies, and the **Algorithmic Logic** governing the AI-driven phytopathology (plant disease) and poultry disease diagnostic engines.

3.2 Software Development Life Cycle (SDLC): The Agile Model

The development of Agri-Go followed the **Agile Iterative Model**, specifically leveraging **Scrum** ceremonies. This approach was selected to mitigate risks associated with the integration of different technologies (React, Flask, and SQLite).

- **Requirements Gathering & Analysis:** Identification of functional requirements (e.g., CRUD operations for inventory) and non-functional requirements (e.g., <2s latency for AI inference).
- **Sprints:** Development was partitioned into bi-weekly sprints. Sprint 1 focused on **RESTful API** endpoint hardening; Sprint 2 on **State Management** and UI/UX reactivity; Sprint 3 on **Computer Vision** model deployment and asynchronous image processing.
- **Continuous Integration:** Iterative testing was conducted to ensure that the **Flask-based** diagnostic engine remained decoupled yet accessible to the **React** client via the `fetch` API.

3.3 System Architecture and Technical Stack

Agri-Go implements a **Three-Tier Architectural Pattern**, ensuring a clean separation of concerns between the presentation, logic, and data layers.

3.3.1 Presentation Layer (Frontend)

Built using the **React.js** library (v18+), the frontend utilizes a **Component-Based Architecture**.

- **State Orchestration:** Leveraging the `useState` and `useEffect` hooks for local state synchronization and `useCallback` for memoizing expensive fetch operations to prevent unnecessary re-renders.
- **Responsive Design:** Implemented via **Tailwind CSS** (a utility-first CSS framework), utilizing a **Flexbox** and **Grid** layout system to ensure cross-platform compatibility across mobile viewports and desktop environments.
- **Iconography & UI Components:** Integration of **Lucide-React** for SVG-based vector icons and **Recharts** for the declarative visualization of farmer sales data via **Area Charts**.

3.3.2 Logic Layer (Backend)

The backend is a **Polyglot Service** architecture:

- **Flask (Python):** Serves as the primary **Application Server**. It handles HTTP requests, manages **JWT (JSON Web Token)** based authentication, and executes the **Deep Learning** inference logic.
- **Werkzeug Utility:** Used for secure file handling and the sanitization of uploaded image filenames (e.g., `secure_filename`) before storage in the `static/uploads` directory.

3.3.3 Data Layer (Persistence)

- **SQLite3:** A lightweight, disk-based Relational Database Management System (RDBMS).
- **Concurrency Handling:** The backend utilizes the `sqlite3.Row` factory to map SQL result sets into Python dictionaries, facilitating seamless **JSON Serialization** for consumption by the React client.

3.4 System Design and Data Modeling

3.4.1 Database Schema (Relational Design)

The database follows the **Third Normal Form (3NF)** to ensure data integrity and minimize redundancy. Key entities include:

- **Users Table:** Stores hashed credentials and farm metadata (Maize vs. Poultry).
- **Produce Table:** Implements a **Foreign Key** relationship (`farmer_id`) linked to the Users table, supporting one-to-many cardinality.
- **Orders Table:** Captures transactional snapshots, including `produce_id`, `buyer_metadata`, and `total_price` calculated as **price** multiplied by the **quantity**. (Total Price = $P * Q$)

3.4.2 Use Case Analysis

The system identifies two primary **Actors**:

1. **Farmer (Producer):** Authorized to perform **C.R.U.D** (Create, Read, Update, Delete) on produce items and trigger the **AI Diagnostic Pipeline**.
2. **Buyer (Consumer):** Authorized to query the marketplace via **String Pattern Matching** (search) and initiate **Order POST requests**.

3.5 AI Diagnostic Engine: Computer Vision Pipeline

The "Disease Detector" module utilizes **Convolutional Neural Networks (CNN)** for image classification.

- **Image Pre-processing:** Uploaded samples undergo **Normalization** and **Resizing** to a standard 224 x 224 pixel tensor.
- **Inference Pipeline:**
 1. The client sends a `multipart/form-data` POST request containing the image.
 2. The Flask server loads a pre-trained model (e.g., **MobileNetV2** or **ResNet50**) optimized for mobile-edge latency.

3. The model outputs a **Softmax Probability Distribution** across disease classes (e.g., *Coccidiosis*, *Maize Blight*, or *Healthy*).
4. The class with the highest confidence score is returned as a JSON object containing the `condition` and `prescriptive_advice`.

3.6 Implementation Security

To protect the integrity of the marketplace, the following security measures were implemented:

- **Input Sanitization:** Guarding against **SQL Injection** via parameterized queries (`cursor.execute(sql, (params,))`).
- **File Validation:** Restricting uploads to specific **MIME types** (JPEG, PNG) to prevent remote code execution.
- **Statelessness:** The API remains stateless, relying on the client to pass identifying metadata (e.g., `user.fullname`) to fetch personalized dashboard metrics.

3.7 System Flow Logic (Process Modeling)

The operational logic of Agri-Go is divided into two primary pipelines: the **Transactional Pipeline** and the **Diagnostic Pipeline**.

3.7.1 The Diagnostic Pipeline (AI Scanner)

This describes the asynchronous flow of image data from the farmer's camera to the Flask inference engine:

1. **Capture:** The Farmer interface (React) captures an image via `input type="file"`.
2. **Encapsulation:** The image is wrapped in a `FormData` object to support `multipart/form-data` encoding.
3. **Transmission:** An asynchronous `POST` request is dispatched to the `/api/identify-crops` or `/api/identify-poultry` endpoint.
4. **Preprocessing:** The Flask backend saves the file using `secure_filename()` and loads it into a **NumPy array**.

5. **Inference:** The CNN model processes the array, calculating a confidence score $C \in [0, 1]$ for various disease classes.
6. **Response:** The class with $\max(C)$ is mapped to a prescriptive advice string and returned as a **JSON payload**.

3.7.2 The Transactional Pipeline (Marketplace)

1. **Listing:** The Farmer submits produce metadata (Price, Unit, Quantity).
2. **Persistence:** The Backend executes an `INSERT INTO produce` query, assigning a **UUID** or auto-incremented ID.
3. **Discovery:** The Buyer client performs a `GET /api/produce` request. The React state (`listings`) is populated via the `useEffect` hook.
4. **Order Initiation:** The Buyer submits an order. The system calculates the `total_price` ($\text{\$Price} \times \text{Quantity}$) and updates the `orders` table.

3.8 Data Dictionary (Database Schema Specification)

This section provides a structural breakdown of the **SQLite** relational schema.

Table 3.1: users Table

Column Name	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY, AUTOINC	Unique identifier for each user
fullname	TEXT	NOT NULL	User's full name (used as Farmer ID)
email	TEXT	UNIQUE, NOT NULL	Primary login identifier

Column Name	Data Type	Constraint	Description
password	TEXT	NOT NULL	Hashed security credential
location	TEXT	NOT NULL	Geographical base of the farm
farm_type	TEXT	NOT NULL	Enum: ['poultry', 'crops']

Table 3.2: produce Table

Column Name	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique item identifier
name	TEXT	NOT NULL	Common name of produce (e.g., Maize)
price	REAL	NOT NULL	Unit price in Ghanaian Cedis (¢)
quantity	INTEGER	DEFAULT 0	Current stock level
unit	TEXT	NOT NULL	Measurement (e.g., Bags, Kilo, Crates)
image_url	TEXT	NOT NULL	Relative path to stored image file
farmer_id	TEXT	FOREIGN KEY	Reference to <code>users.fullname</code>

Table 3.3: orders Table

Column Name	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique transaction ID
produce_id	INTEGER	FOREIGN KEY	Link to the specific produce item
buyer_name	TEXT	NOT NULL	Name of the person placing the order
total_price	REAL	NOT NULL	Computed cost (\$Price \times Qty\$)
status	TEXT	DEFAULT 'Pending'	Enum: ['Pending', 'Completed']
order_date	TIMESTAMP	DEFAULT CURRENT	ISO 8601 formatted timestamp

3.9 System Security and Validation Protocols

To ensure the robustness of the **Agri-Go** ecosystem, the methodology incorporates the following validation layers:

- **Client-Side Validation:** React forms utilize controlled components to prevent the submission of null values or negative prices.
- **Server-Side Sanitization:** The Flask backend utilizes **parameterized SQL queries** to prevent **SQL Injection (SQLi)** attacks.
- **MIME-Type Filtering:** Image uploads are restricted to `.jpg`, `.jpeg`, and `.png` extensions to mitigate the risk of shell script execution.

3.10 Chapter Summary

In conclusion, Chapter Three has provided a comprehensive technical roadmap of the methodology employed in the architectural design and engineering of the **Agri-Go** platform. By adopting an **Agile Iterative Model**, the project ensured the seamless integration of a high-performance **React.js** frontend with a robust **Flask-based** Python backend.

The chapter elucidated the system's **Three-Tier Architecture**, emphasizing the separation of concerns between the presentation layer, the application logic, and the **SQLite** data persistence layer. Furthermore, the technical specifications of the **Computer Vision pipeline** were detailed, highlighting the transition from raw image ingestion to AI-driven diagnostic inference. Through the use of **Data Dictionaries** and **Process Modeling**, the chapter established a rigorous framework for data integrity and system security. This methodological foundation serves as the prerequisite for the subsequent implementation, testing, and performance evaluation phases detailed in Chapter Four.